Introdução à Computação
tipos de dados em C, entrada e saída

Alexandre Rademaker

variáveis, literais e tipos I

Variáveis são nomes para localizações na memória que poderão conter valores e um valor é sempre de um determinado **tipo**.

O tipo só precisa ser especificado na declaração da variável. São **palavras reservadas**.

Literais são notações para representar um valor.

```
1 int a = 12, b;
2 double pi = 3.14;
3 float b = 1.3E2;
4
5 char test;
6 test = 'a';
```

variáveis, literais e tipos II

Em C, cada tipo de dado é armazenado em uma certa quantidade de bytes.

bool	1 byte
char	1 byte
short	2 bytes
int	4 bytes (32 bits) que podem representar até 2^{32} valores diferentes.
long	8 bytes
float	4 bytes
double	8 bytes

entre outros ...

variáveis, literais e tipos III

Os identicadores (nomes) não pode ser uma **palavra reservada** (`int`, `true`, `if`, `else` etc).

São formados por uma combinação de letras, dígitos e '_' (underline), começando sempre com uma letra ou '_'.

Letras maiúsculas e minúsculas são diferentes. para assegurar a portabilidade use no máximo 31 caracteres. escolha nomes **significativos** para clareza do código.

variáveis, literais e tipos IV

Alguns erros comuns:

```
1 int var1, 2var, _var3;
2 int int;
3 int double;
```

Variáveis podem ser declaradas em qualquer lugar de um programa C, mas devem aparecer antes de serem usadas no programa. Variáveis tem um escopo de vida, veremos mais à frente.

variáveis, literais e tipos V

```
1 char c = 'a';
2 int b = 2630692;
```

endereço	valor
0x10000	01100001
0x10001	?
0x10002	?
0x10003	00000000
0x10004	00101000
0x10005	00100100
0x10006	00100100
0x10007	?
0x10008	?
0x10009	?
0x10010	?

Variáveis tem nome, valor e endereço (acessível via `&c`)

operador de atribuição

```
1 s = 10 + n;
```

- O = é atribuição, não teste de igualdade. O + é a adição. Ambos binários.
- A expressão à direita é avaliação e seu resultado atribuído a variável s.
- Sempre usar espaços entre operadores e operandos, ajuda a leitura.

operadores aritméticos

operação	operador
precedência	( )
multiplicação	*
soma	+
subtração	-
divisão	/
resto / módulo	%

```
int a = (12 + 4) * 4 % 2;
```

entrada e saída

Um programa precisa receber dados para processar (entrada) e produzir uma saída.

Podemos passar dados por parâmetros na linha de comando, ler de um arquivo ou solicitar interativamente ao usuário.

Para produzir dados, podemos imprimir na tela ou gravar dados em um arquivo.

Entre outras formas!

a função printf I

A função `printf` é parte da biblioteca `<stdio.h>`.

```
printf(<format-string>, e1, e2, ...)
```

onde $e1, e2, \ldots$ são expressão pode ser: um nome de variável, um literal ou expressões envolvendo nomes e literais.

```
1 printf("hello world"); // certo
2
3 printf(12); // erro
4 int v = 12;
5 print(v); // erro
6
7 printf("%d", v); // certo
```

a função `printf` II

Alguns possíveis 'especificadores' para o comando printf:

- `%d` int (número inteiro)
- `%ld` long long (número inteiro)
- `%f` float (ponto flutuante)
- `%lf` double (ponto flutuante)
- `%c` char (caractere)
- `%s` string (cadeia de caracteres)

a função printf III

```
1 int a = 10;
2 int b = 20;
3 printf("Soma de %d e %d = %d\n", a, b, a + b);
```

Irá produzir

```
Soma de 10 e 20 = 30
```

a função printf IV

```
1 #include <stdio.h>
2
3 int main() {
4   printf("%-3s %8s\n"  , "Var", "Val");
5   printf("%-3s %8.2f\n", "x"  , 10.222);
6   printf("%-3s %8.2f\n", "y"  , 20.33);
7   printf("%-3s %8.2f\n", "z"  , 30E2);
8 }
```

```
1 Var       Val
2 x       10.22
3 y       20.33
4 z     3000.00
```

a função printf V

ainda temos os caracteres especias

- \n quebra de linha, passa para a linha debaixo
- \t tabulação horizontal
- \" aspas duplas
- \' aspas simples ou apóstrofo
- \\ barra invertida
- \a beep

a função `scanf` I

A função `scanf` também é parte da biblioteca `<stdio.h>`.

Utilizada para ler da entrada padrão (terminal). O scanf tem algumas (grandes) diferenças em relação ao printf.

- A função printf imprime texto e o **valor** de variáveis
- A função scanf **altera o conteúdo** das variáveis
- Alterar conteúdo equivale a **modificar o que está na memória**. Por esta razão, sempre passamos um **endereço de memória** para a função scanf.

a função scanf II

scanf(<format-string>, addr1, addr2, ...)

onde addr1, addr2,... são endereços de memória.

```
1 int x;
2 scanf("%d", &x);
3
4 scanf(x); // erro
5 scanf("%d", x); // erro
```

a função scanf III

```c
int n;
float m;
char c;
scanf("%d ",  &n);
scanf("%c ",  &c);
scanf("/%f/", &m);
printf("n:[%d] c:[%c] m:[%.3f] \n", n, c, m);
```

```
> clang test.c && ./a.out
123   a    /4.5/
n:[123] c:[a] m:[4.500]
```

Veja `man 3 scanf`

exemplos I

Vamos fazer uma média de 3 valores? O que iremos ver na saída?

src/scores0.c

```c
#include <stdio.h>

int main(void) {

    int score1 = 72;
    int score2 = 73;
    int score3 = 33;

    printf("Average: %d\n",
     (score1 + score2 + score3) / 3.0);
}
```

exemplos II

```
ar@tenis src % make scores0
cc  -I/usr/local/opt/openjdk/include   scores0.c   -o scores0
scores0.c:9:29: warning: format specifies type 'int' but the argument has type 'double' [-Wformat]
    printf("Average: %d\n", (score1 + score2 + score3) / 3.0);
                     ~~     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
                     %f
1 warning generated.
ar@tenis src % ./scores0
Average: -1282558296
ar@tenis src %
```

exemplos III

Soma de 3 inteiros divididos por um inteiro é um inteiro e o resto descartado.

‘printf‘ está esperando um `int` (`%d`) em seu argumento `format string` (string de formatação).

Mudando `%d` para `%f` ... A função ‘printf‘ precisa saber como ler e escrever os bytes!